Task 1: Apache Airflow Environment Setup

1. Apache Airflow Environment Setup

- Objective: Run Airflow using Docker (or fallback on EC2).
- **Steps :**
    - Installed Docker & Docker Compose.
    - Downloaded official docker-compose.yaml.
    - Created dags/, logs/, plugins/, and .env with AIRFLOW_UID.
    - Ran docker compose up airflow-init then docker compose up -d.
    - Accessed Airflow UI at http://localhost:8080 (default login: airflow/airflow).

STEP:

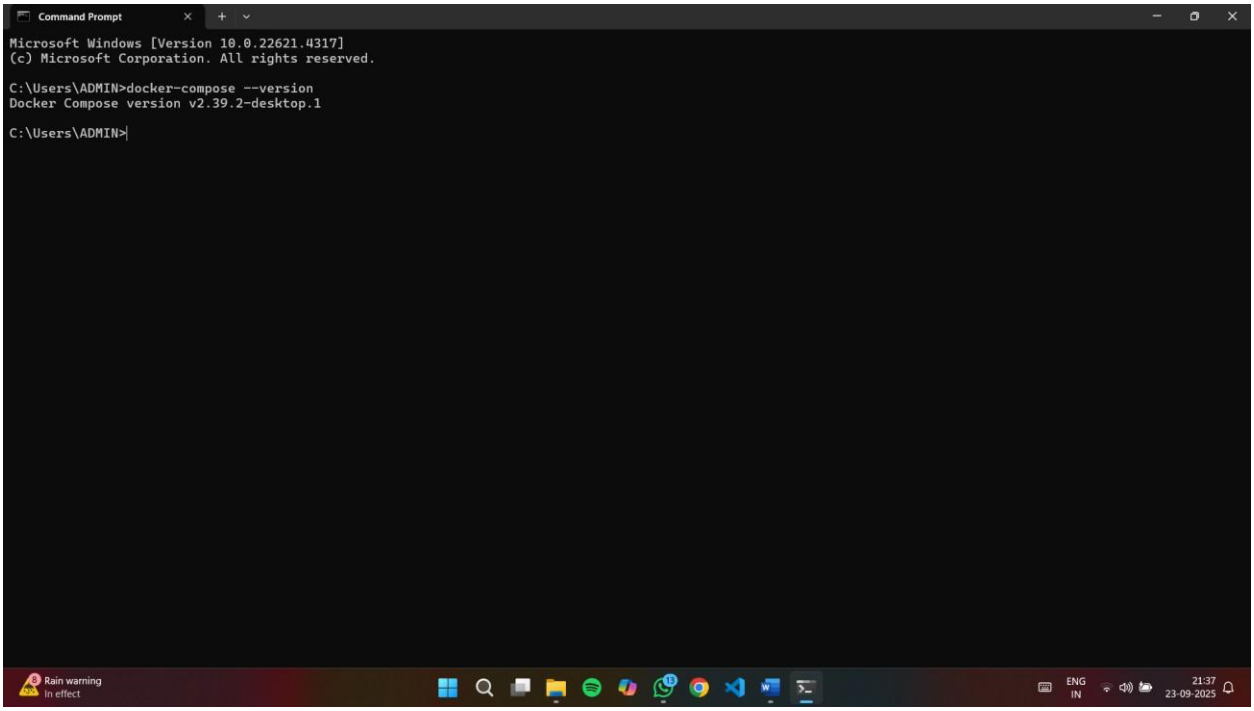
1. Install Prerequisites

On Local Machine (Linux/Mac/Windows WSL):

```
# Install Docker
sudo apt update && sudo apt install -y docker.io docker-compose

# Verify installation
docker --version
docker-compose --version
```

On EC2 (Ubuntu):

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
sudo apt install -y docker.io docker-compose

# Enable Docker at startup
sudo systemctl enable docker
sudo systemctl start docker
```

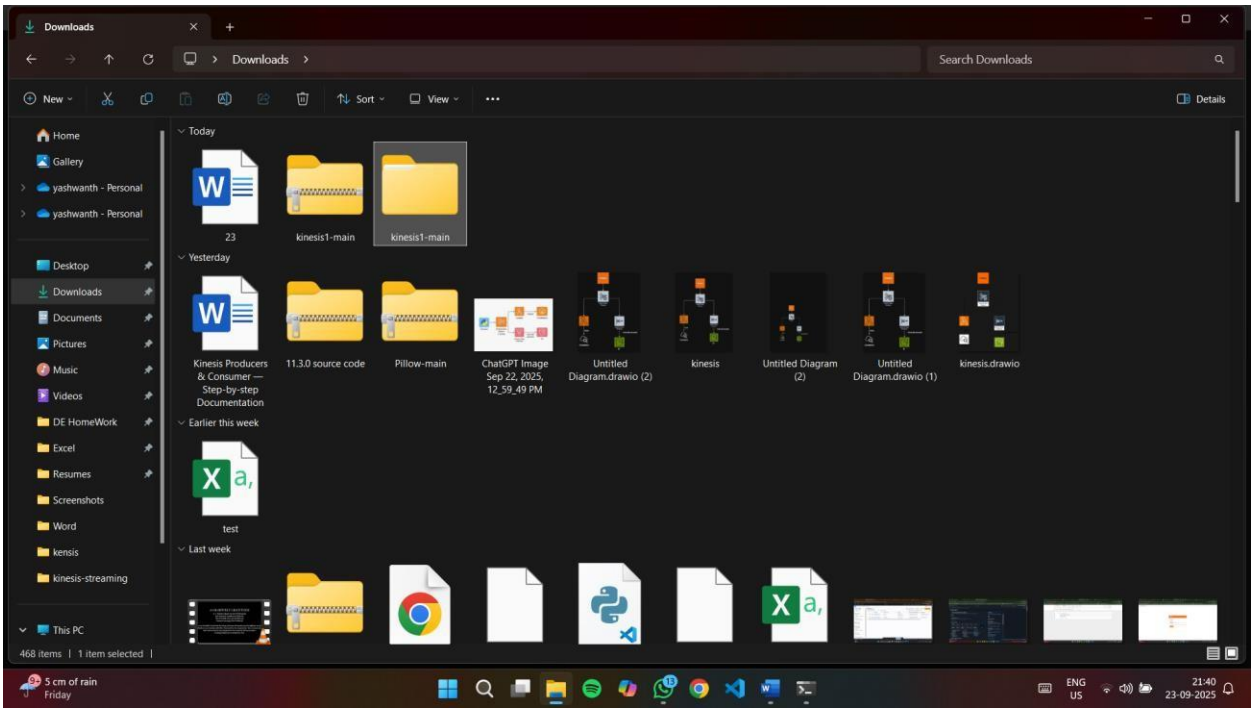


2. Project Setup

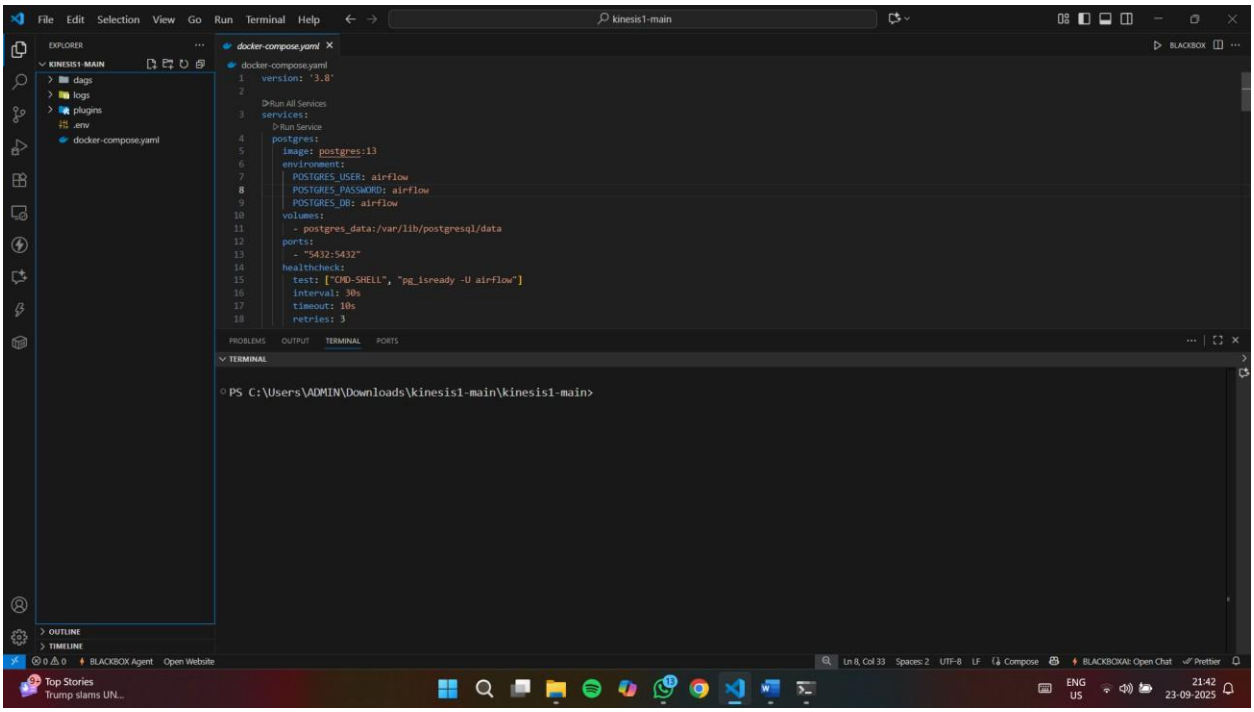
Clone repo from github:

https://github.com/karthick180887/kinesis1

Extracxt the zip file



4. Open in Vscode



docker-compose.yaml

```
version: '3.8'
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U airflow"]
      interval: 30s
      timeout: 10s
      retries: 3

  airflow-init:
    image: apache/airflow:2.7.1
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
      AIRFLOW__CORE__FERNET_KEY: ''
```

```
      AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
      AIRFLOW__CORE__LOAD_EXAMPLES: 'true'
      AIRFLOW__WEBSERVER__SECRET_KEY: airflow-secret-key
      _AIRFLOW_DB_UPGRADE: 'true'
      _AIRFLOW_WWW_USER_CREATE: 'true'
      _AIRFLOW_WWW_USER_USERNAME: airflow
      _AIRFLOW_WWW_USER_PASSWORD: airflow
    volumes:
      - ./dags:/opt/airflow/dags
      - ./logs:/opt/airflow/logs
      - ./plugins:/opt/airflow/plugins
    depends_on:
      postgres:
        condition: service_healthy
    command: version

  airflow-webserver:
    image: apache/airflow:2.7.1
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
      AIRFLOW__CORE__FERNET_KEY: ''
      AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
      AIRFLOW__CORE__LOAD_EXAMPLES: 'true'
      AIRFLOW__WEBSERVER__SECRET_KEY: airflow-secret-key
    volumes:
      - ./dags:/opt/airflow/dags
      - ./logs:/opt/airflow/logs
      - ./plugins:/opt/airflow/plugins
    ports:
      - "8080:8080"
    command: webserver
    depends_on:
      - airflow-init
      - postgres
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5

  airflow-scheduler:
    image: apache/airflow:2.7.1
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
```

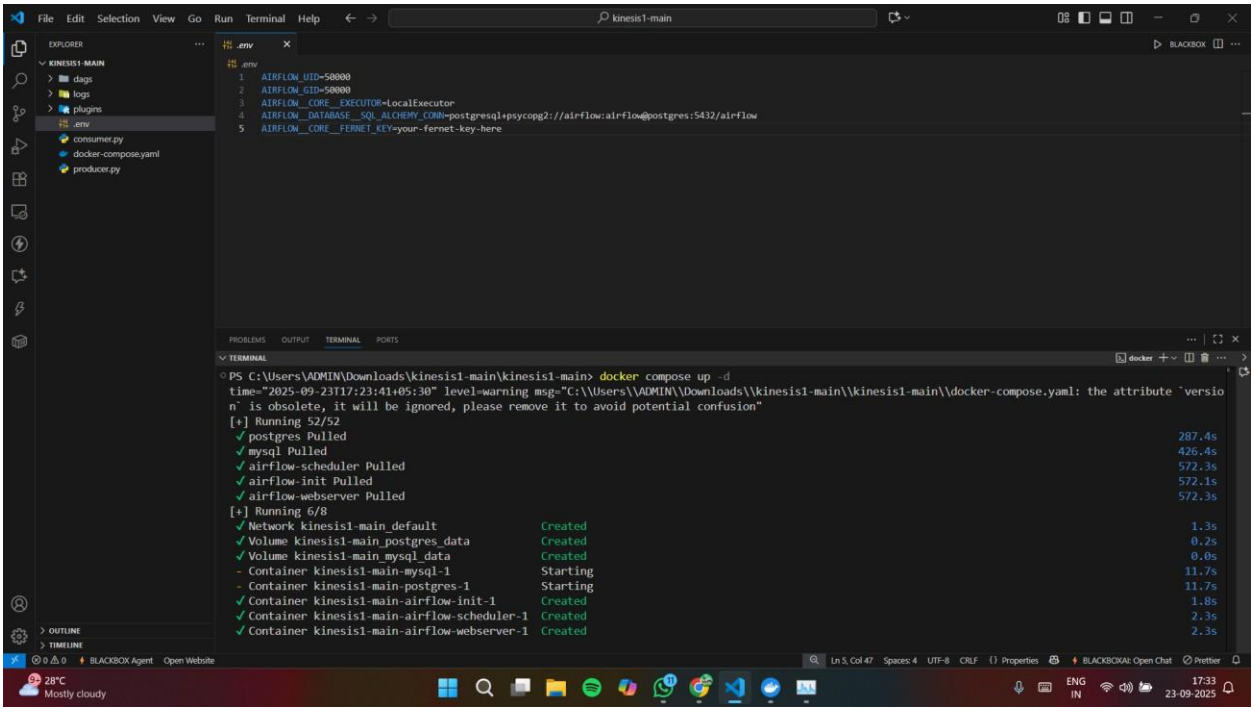
```
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__LOAD_EXAMPLES: 'true'
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins
  command: scheduler
  depends_on:
    - airflow-init
    - postgres

volumes:
  postgres_data:
```

.env

```
AIRFLOW_UID=50000
AIRFLOW_GID=50000

AIRFLOW CORE EXECUTOR=LocalExecutor
AIRFLOW DATABASE SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airf
low@postgres:5432/airflow
AIRFLOW CORE FERNET_KEY=your-fernet-key-here
```

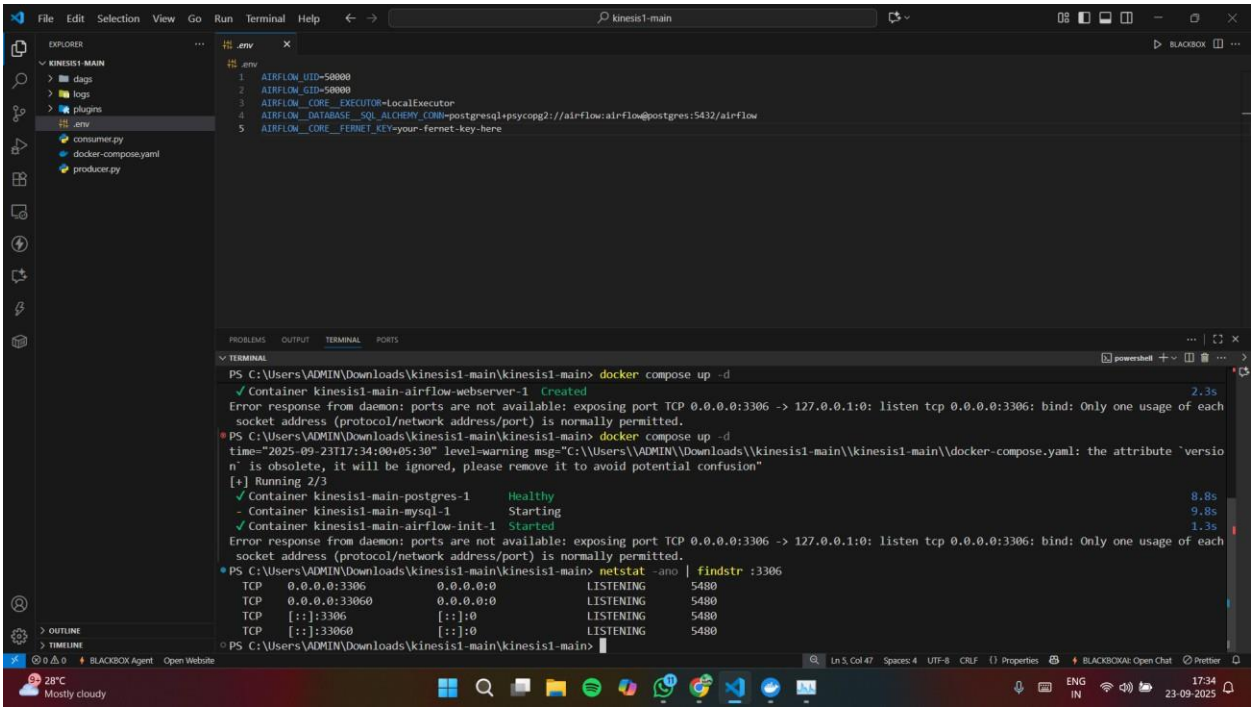
5. Pull Image

docker-compose up -d



Run the container

docker compose up -d



Error:

Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:3306 -> 127.0.0.1:0:

listen tcp 0.0.0.0:3306: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.

This means something on your Windows system is already using port 3306 (the default MySQL port).

1. Check who is using port 3306

   Run in PowerShell:

   netstat -ano | findstr :3306

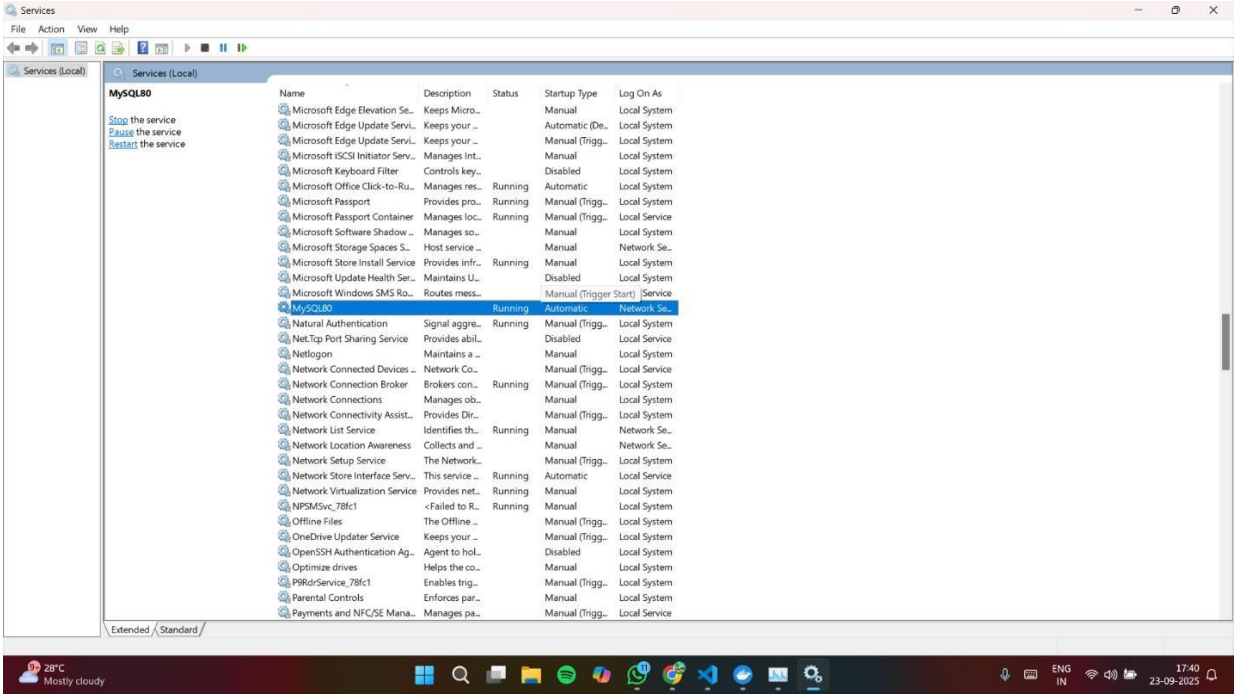
   This will show the PID of the process using 3306.

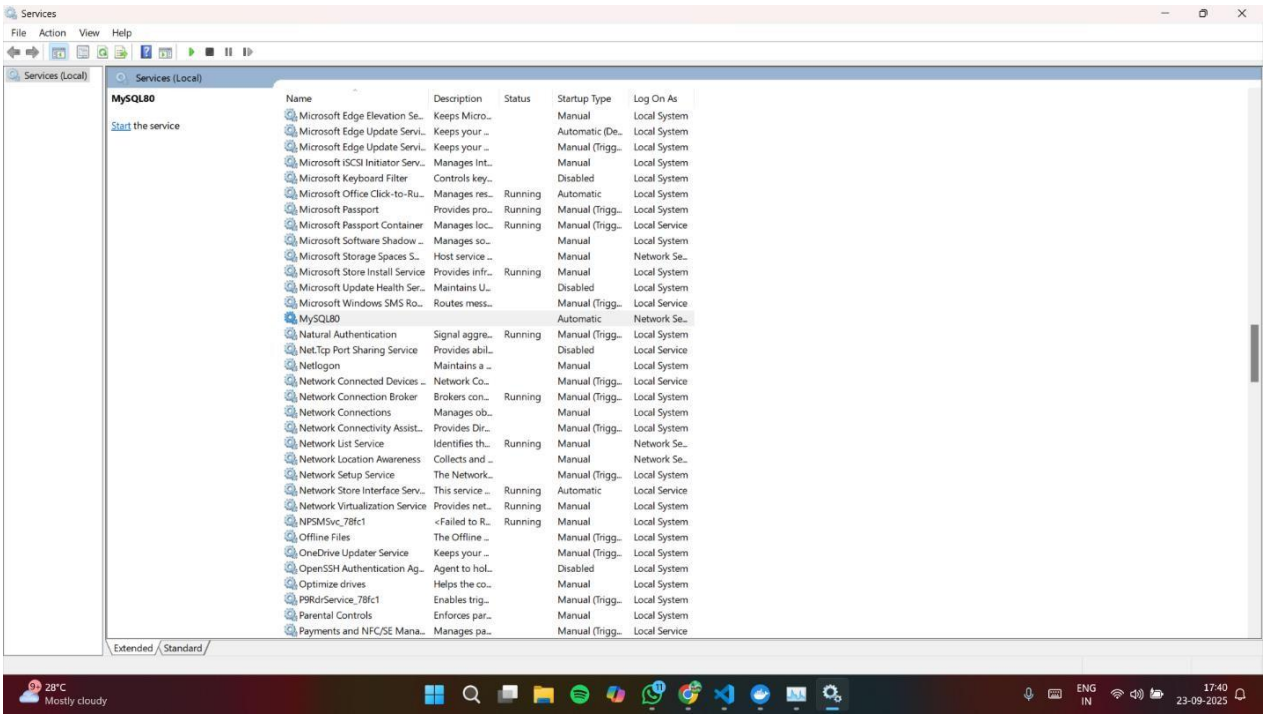
2. Stop Local MySQL:

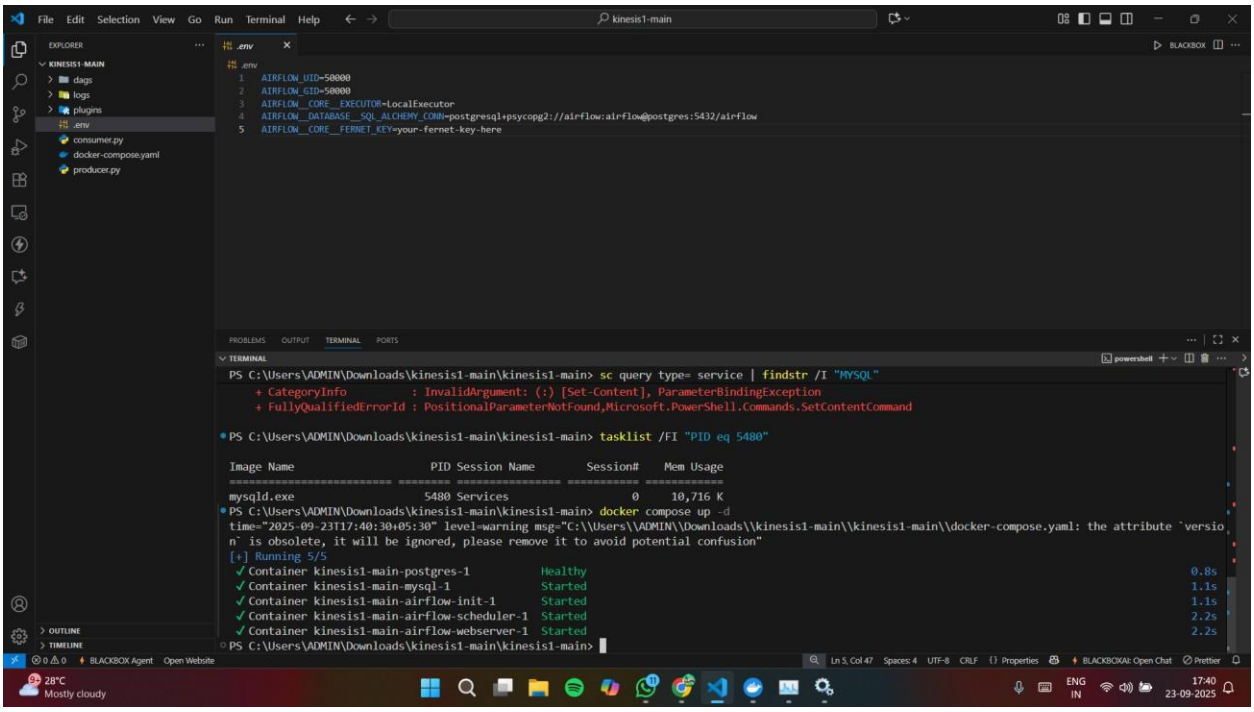
   To stop a local MySQL server, which will free up the port it is using (defaulting to 3306), the method depends on your operating system and how MySQL was installed.

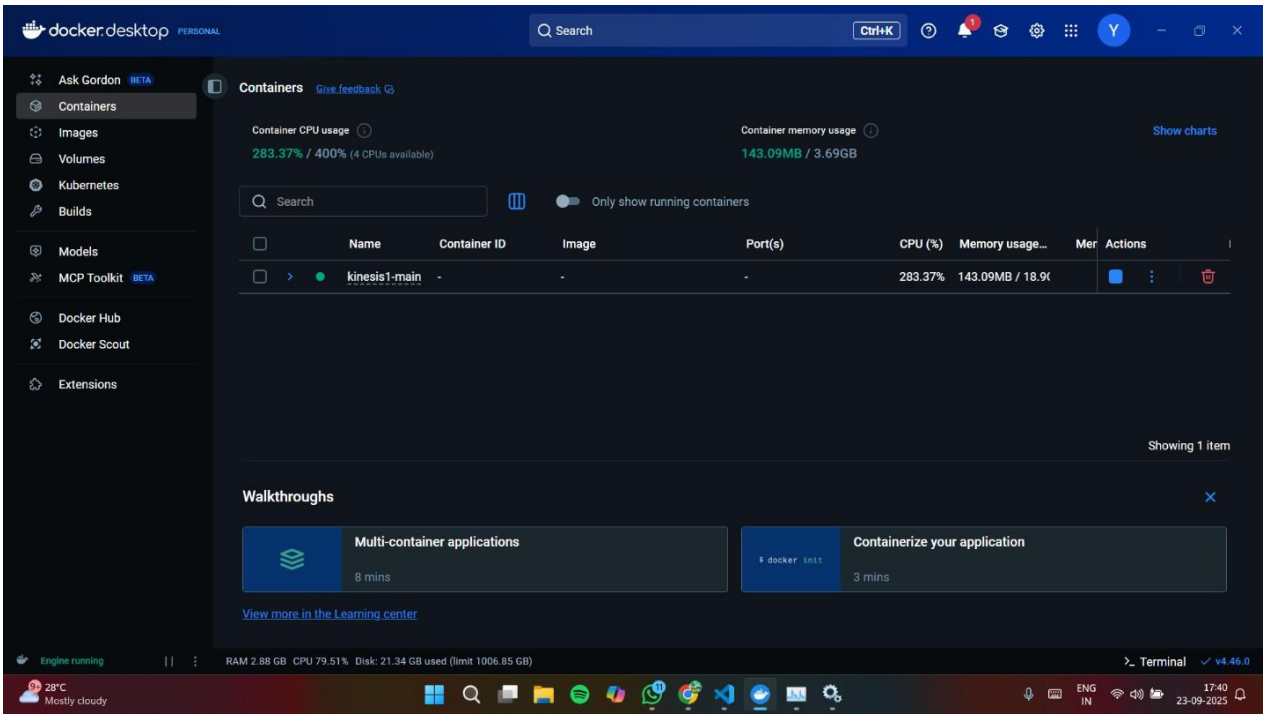
On Windows:

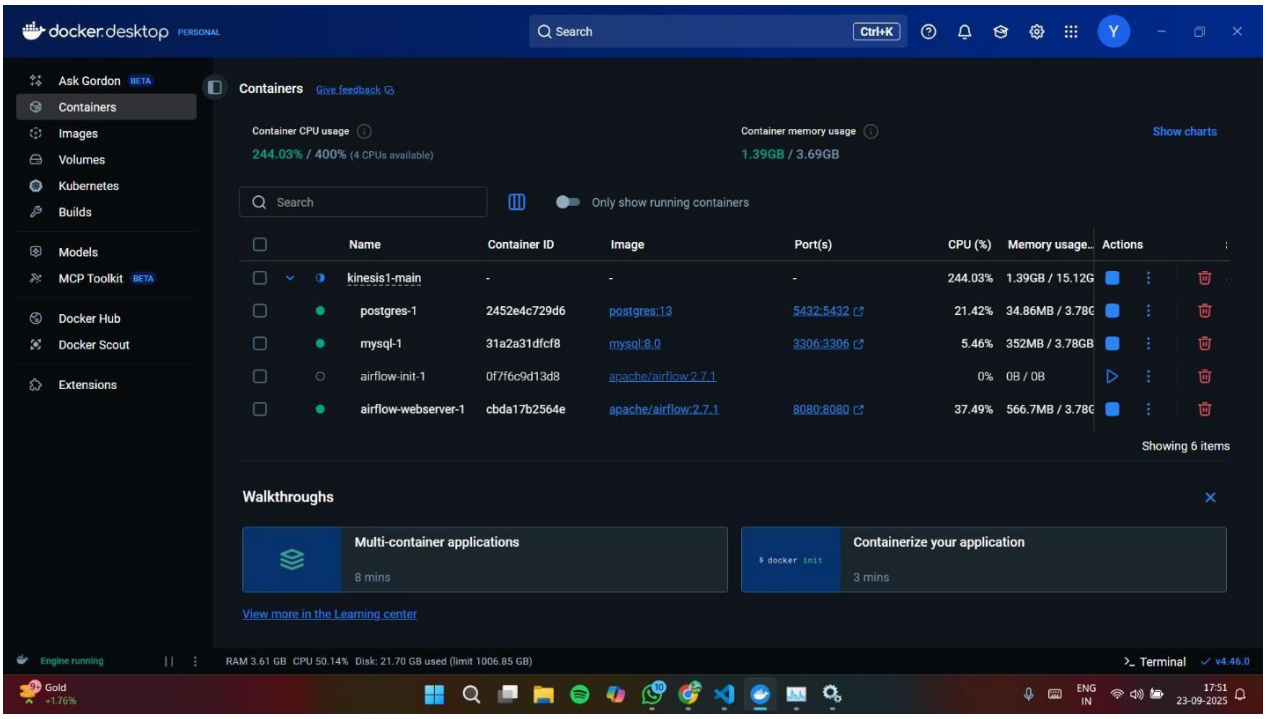
- Using Services:
- Open the "Run" dialog by pressing Win + R.
- Type services.msc and press Enter.
- Locate the MySQL service (e.g., "MySQL80" for MySQL 8.0).
- Right-click on the service and select "Stop."




Services
File Action View Help
Services (Local)
MySQL80
Start the service
Name Description Status Startup Type Log On As
Microsoft Edge Elevation Se... Keeps Micro... Manual Local System
Microsoft Edge Update Servi... Keeps your ... Automatic (De... Local System
Microsoft Edge Update Servi... Keeps your ... Manual (Trigg... Local System
Microsoft iSCSI Initiator Serv... Manages Int... Manual Local System
Microsoft Keyboard Filter Controls key... Disabled Local System
Microsoft Office Click-to-Ru... Manages res... Running Automatic Local System
Microsoft Passport Provides pro... Running Manual (Trigg... Local System
Microsoft Passport Container Manages loc... Running Manual (Trigg... Local Service
Microsoft Software Shadow ... Manages so... Manual Local System
Microsoft Storage Spaces S... Host service ... Manual Network Se...
Microsoft Store Install Service Provides infr... Running Manual Local System
Microsoft Update Health Ser... Maintains U... Disabled Local System
Microsoft Windows SMS Ro... Routes mess... Manual (Trigg... Local Service
MySQL80 Automatic Network Se...
Natural Authentication Signal aggre... Running Manual (Trigg... Local System
Net.Tcp Port Sharing Service Provides abil... Disabled Local Service
Netlogon Maintains a ... Manual Local System
Network Connected Devices ... Network Co... Manual (Trigg... Local Service
Network Connection Broker Brokers con... Running Manual (Trigg... Local System
Network Connections Manages ob... Manual Local System
Network Connectivity Assist... Provides Dir... Manual (Trigg... Local System
Network List Service Identifies th... Running Manual Network Se...
Network Location Awareness Collects and ... Manual Network Se...
Network Setup Service The Network... Manual (Trigg... Local System
Network Store Interface Serv... This service ... Running Automatic Local Service
Network Virtualization Service Provides net... Running Manual Local System
NPSMSvc_78fc1 <Failed to R... Running Manual Local System
Offline Files The Offline ... Manual (Trigg... Local System
OneDrive Updater Service Keeps your ... Manual (Trigg... Local System
OpenSSH Authentication Ag... Agent to hol... Disabled Local System
Optimize drives Helps the co... Manual Local System
P9RdrService_78fc1 Enables trig... Manual (Trigg... Local System
Parental Controls Enforces par... Manual Local System
Payments and NFC/SE Mana... Manages pa... Manual (Trigg... Local Service
Extended Standard
28°C Mostly cloudy
ENG IN 17:40 23-09-2025


File Edit Selection View Go Run Terminal Help
kinesis1-main
EXPLORER
KINESIS1-MAIN
dags
logs
plugins
.env
consumer.py
docker-compose.yaml
producer.py
.env
AIRFLOW_UID=50000
AIRFLOW_GID=50000
AIRFLOW__CORE__EXECUTOR=LocalExecutor
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
AIRFLOW__CORE__FERNET_KEY=your-fernet-key-here
PROBLEMS OUTPUT TERMINAL PORTS
PS C:\Users\ADMIN\Downloads\kinesis1-main\kinesis1-main> sc query type= service | findstr /I "MYSQL"
+ CategoryInfo : InvalidArgument: (:) [Set-Content], ParameterBindingException
+ FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.SetContentCommand
PS C:\Users\ADMIN\Downloads\kinesis1-main\kinesis1-main> tasklist /FI "PID eq 5480"
Image Name PID Session Name Session# Mem Usage
mysqld.exe 5480 Services 0 10,716 K
PS C:\Users\ADMIN\Downloads\kinesis1-main\kinesis1-main> docker compose up -d
time="2025-09-23T17:40:30+05:30" level=warning msg="C:\\Users\\ADMIN\\Downloads\\kinesis1-main\\kinesis1-main\\docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 5/5
Container kinesis1-main-postgres-1 Healthy 0.8s
Container kinesis1-main-mysql-1 Started 1.1s
Container kinesis1-main-airflow-init-1 Started 1.1s
Container kinesis1-main-airflow-scheduler-1 Started 2.2s
Container kinesis1-main-airflow-webserver-1 Started 2.2s
PS C:\Users\ADMIN\Downloads\kinesis1-main\kinesis1-main>
28°C Mostly cloudy
ENG IN 17:40 23-09-2025



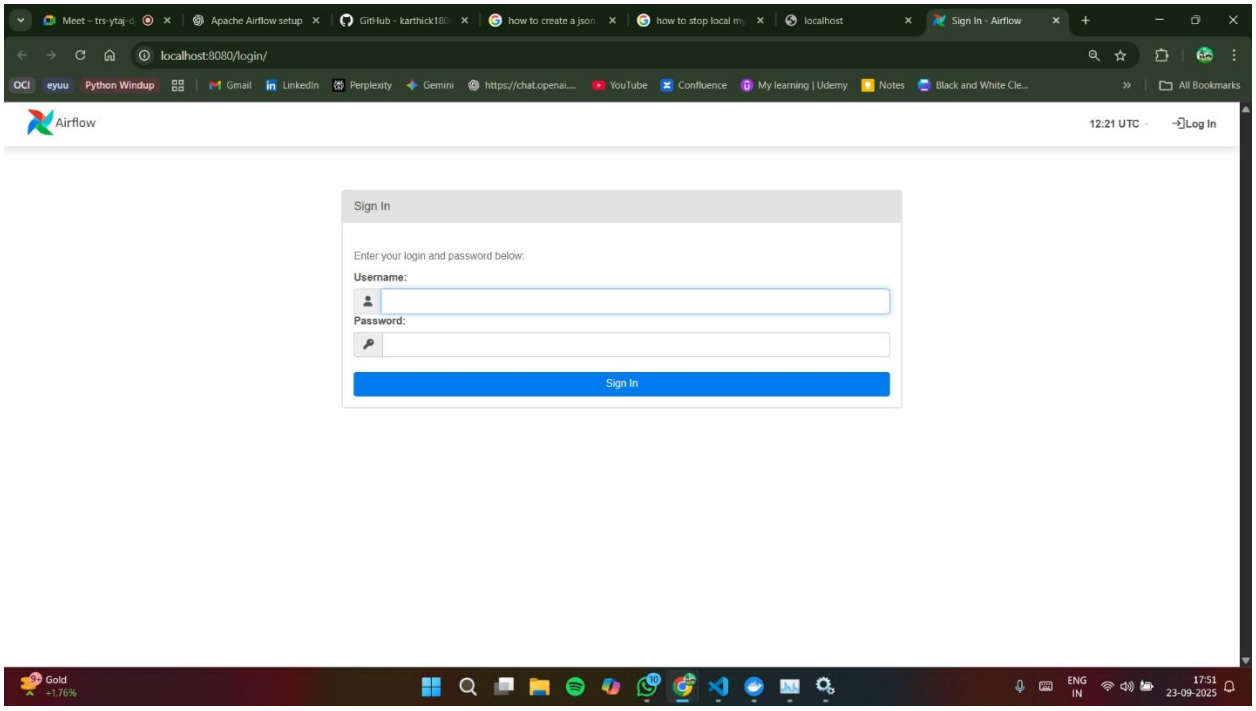


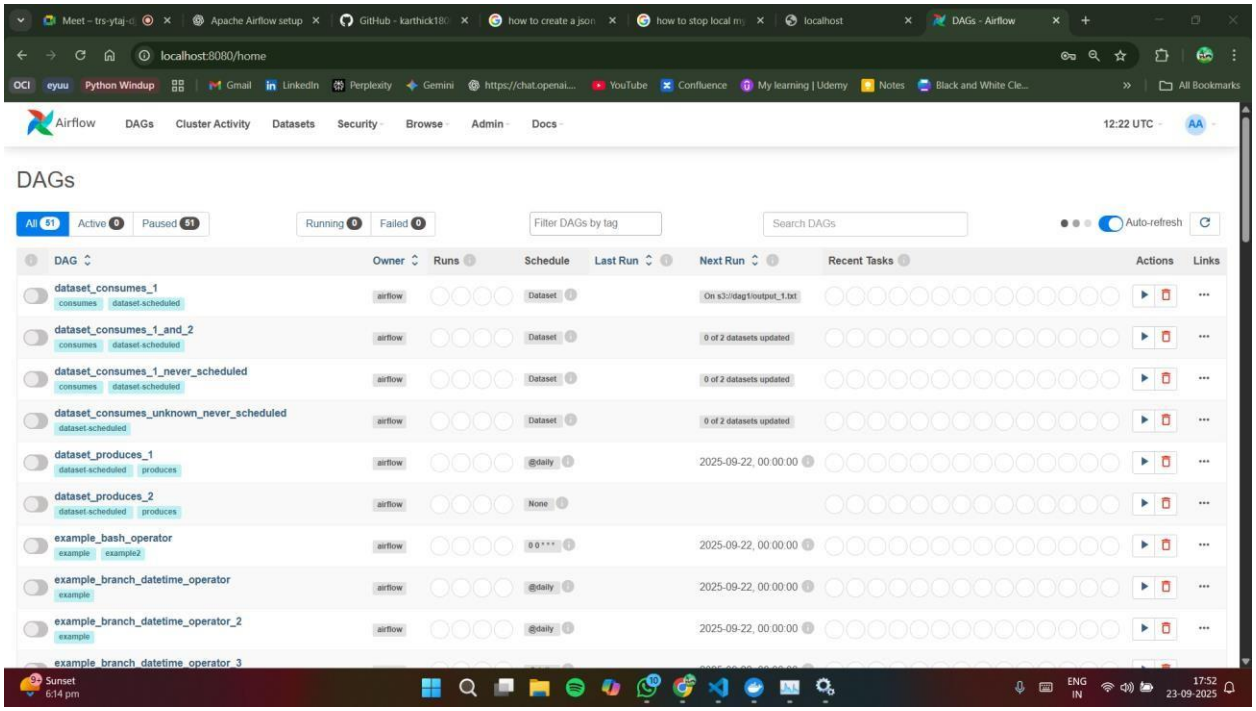
6. Access Airflow

- Local: http://localhost:8080
- EC2: http://<your-ec2-public-ip>:8080

Login with:

- **Username:** airflow
- **Password:** airflow





7. Verify DAGs Directory

Put your DAGs in the dags/ folder. Example:

```
echo "print('Hello Airflow')" > dags/test_dag.py
```

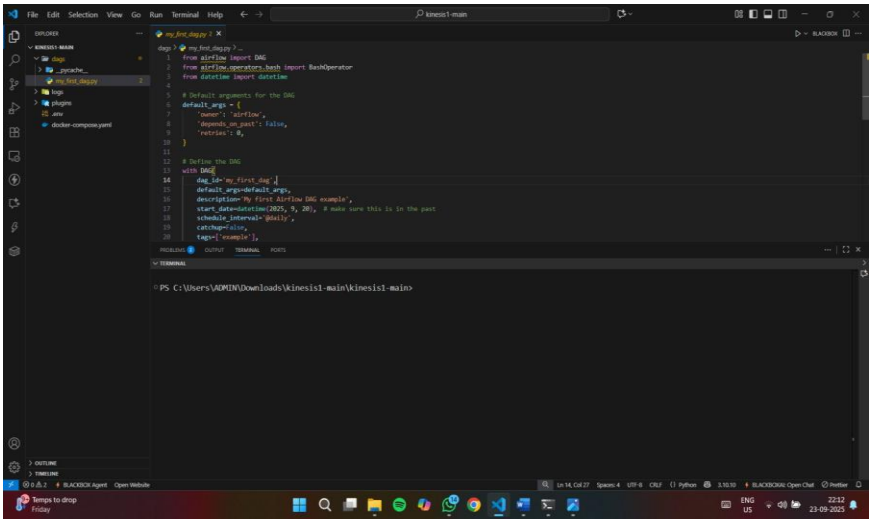
Refresh Airflow UI → DAG should appear.

8. (Optional) Run Producer/Consumer as DAGs

You can wrap your **Kinesis producer/consumer scripts** as Airflow DAGs.

Example dags/kinesis_dag.py:

[redacted]


File Edit Selection View Go Run Terminal Help
kinesis1-main
EXPLORER
KINESIS1-MAIN
dags
__pycache__
my_first_dag.py
logs
plugins
.env
docker-compose.yaml
my_first_dag.py 2
dags > my_first_dag.py > ...
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime
# Default arguments for the DAG
default_args = {
'owner': 'airflow',
'depends_on_past': False,
'retries': 0,
}
# Define the DAG
with DAG(
dag_id='my_first_dag',
default_args=default_args,
description='My first Airflow DAG example',
start_date=datetime(2025, 9, 20), # make sure this is in the past
schedule_interval='@daily',
catchup=False,
tags=['example'],
PROBLEMS 2 OUTPUT TERMINAL PORTS
TERMINAL
PS C:\Users\ADMIN\Downloads\kinesis1-main\kinesis1-main>
OUTLINE
TIMELINE
BLACKBOX Agent Open Website
Ln 14, Col 27 Spaces: 4 UTF-8 CRLF {} Python 3.10.10 BLACKBOXAI: Open Chat Prettier
Temps to drop
Friday
ENG
US
22:12
23-09-2025

Task 2: Your First Airflow DAG

Code for my_first_dag.py

Save this inside your dags/ folder (created during setup):

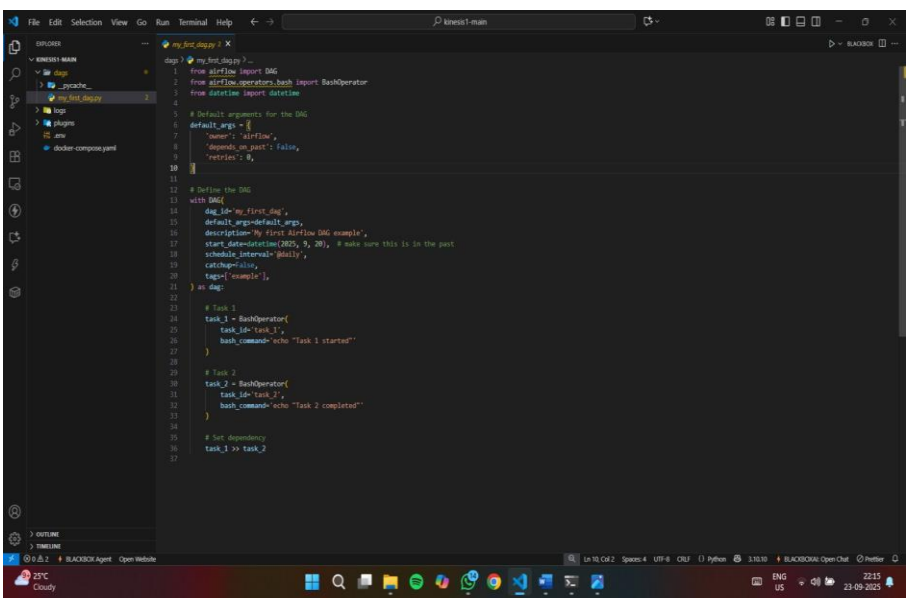
Steps to Run

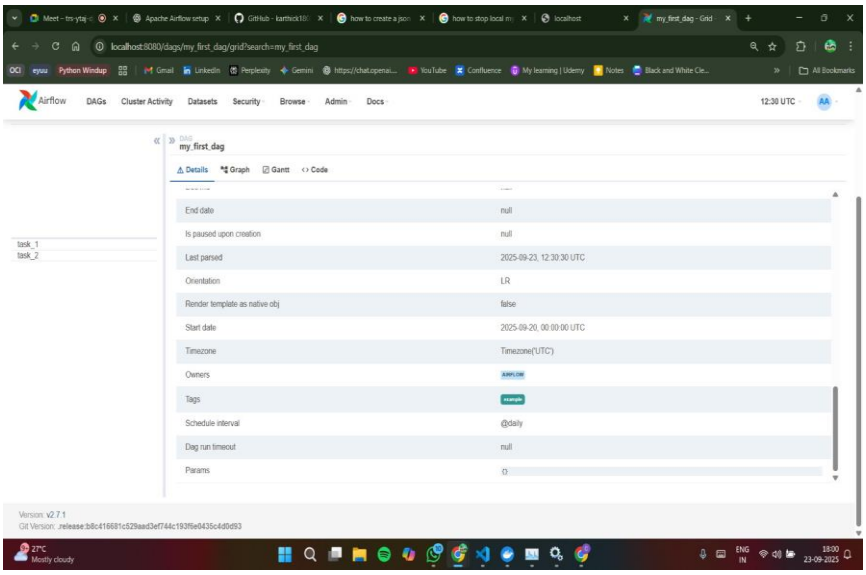
1. Place the file into your dags/ folder:
2. cp my_first_dag.py dags/
3. Restart Airflow containers (so it picks up the new DAG):
4. docker-compose restart airflow-webserver airflow-scheduler
5. Open **Airflow UI**:
    - Local: http://localhost:8080
    - EC2: http://<EC2-Public-IP>:8080

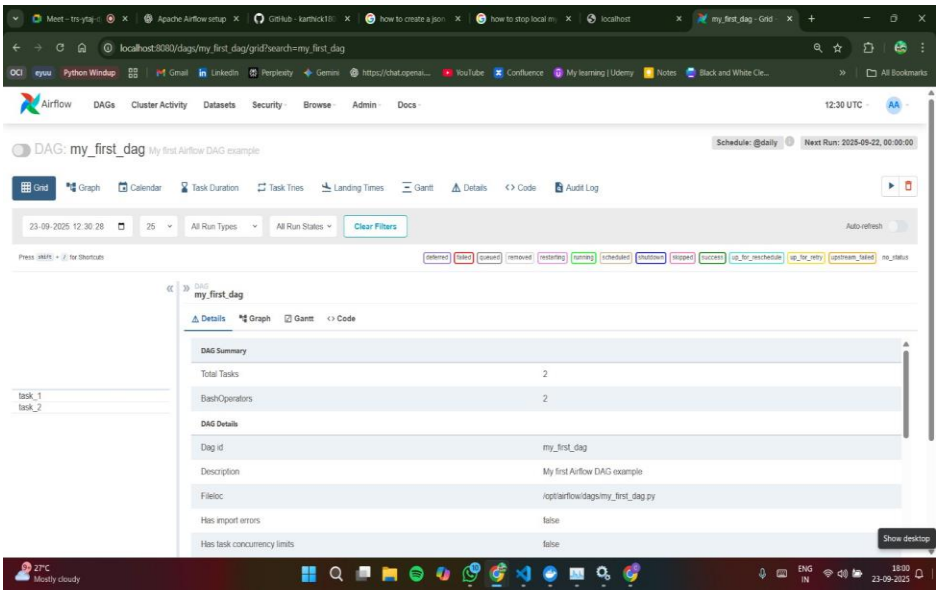
```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

# Default arguments for the DAG
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'retries': 0,
}

# Define the DAG
with DAG(
    dag_id='my_first_dag',
    default_args=default_args,
    description='My first Airflow DAG example',
    start_date=datetime(2025, 9, 20),  # make sure this is in the past
    schedule_interval='@daily',
    catchup=False,
    tags=['example'],
) as dag:
```

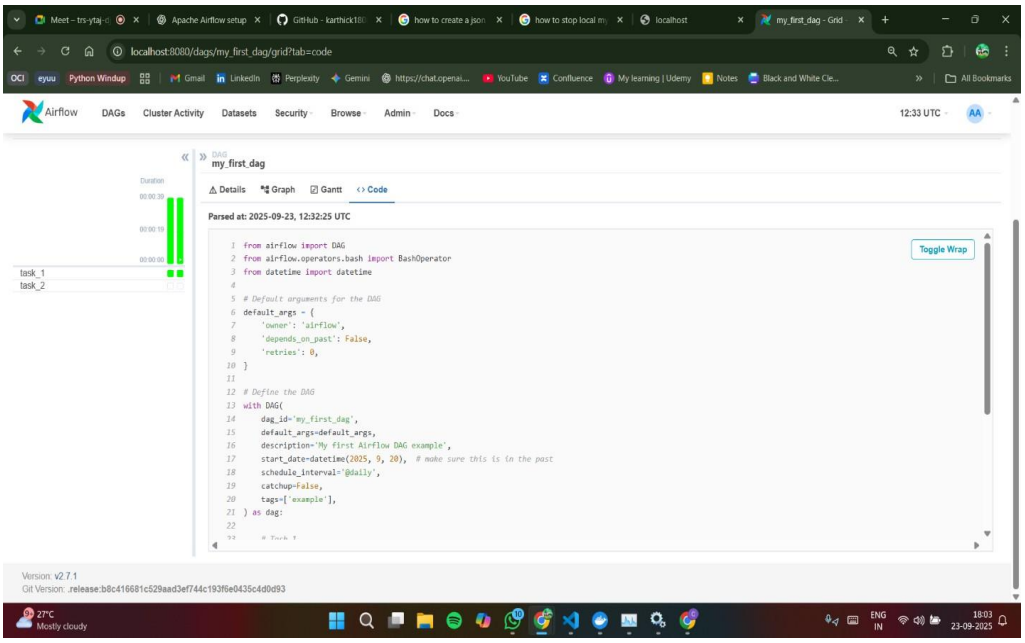
```
# Task 1
task_1 = BashOperator(
    task_id='task_1',
    bash_command='echo "Task 1 started"'
)

# Task 2
task_2 = BashOperator(
    task_id='task_2',
    bash_command='echo "Task 2 completed"'
)

# Set dependency
task_1 >> task_2
```

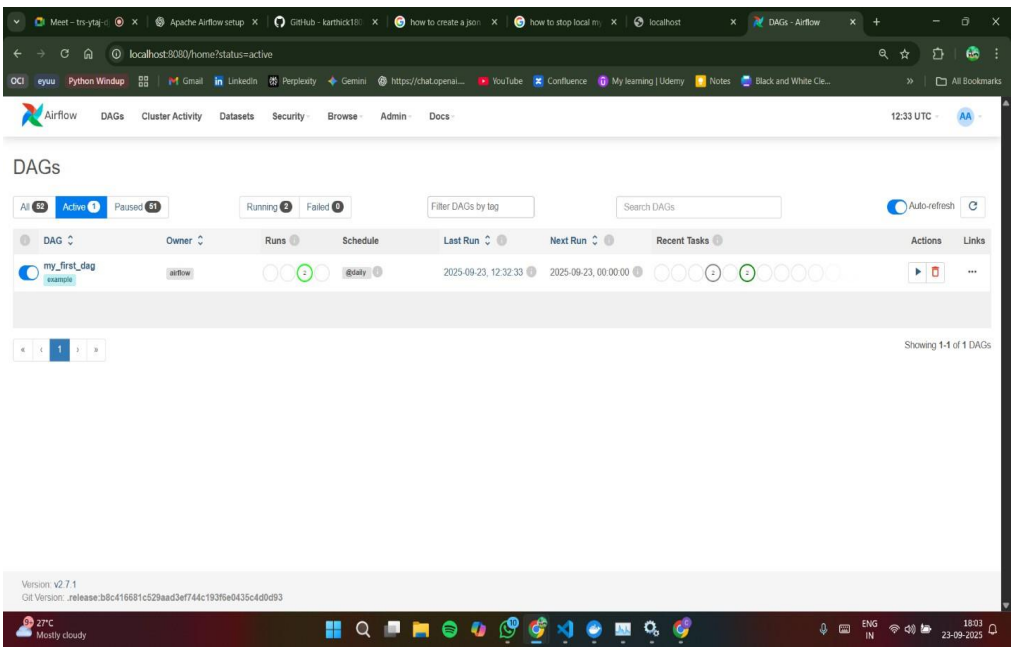
6. Login → You'll see my_first_dag in the DAG list.
7. **Trigger the DAG manually** (click ▶) → Go to **Graph View** → You'll see:
8. task_1 → task_2
9. Both tasks should run successfully (green boxes in Graph view).




localhost:8080/dags/my_first_dag/grid?search=my_first_dag
Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
12:30 UTC
my_first_dag
Details
Graph
Gantt
Code
task_1
task_2
End date
null
Is paused upon creation
null
Last parsed
2025-09-23, 12:30:30 UTC
Orientation
LR
Render template as native obj
false
Start date
2025-09-20, 00:00:00 UTC
Timezone
Timezone('UTC')
Owners
AIRFLOW
Tags
example
Schedule interval
@daily
Dag run timeout
null
Params
{}
Version: v2.7.1
Git Version: .release:b8c416681c529aad3ef744c193f6e0435c4d9d93


localhost:8080/dags/my_first_dag/grid?search=my_first_dag
Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
12:30 UTC
DAG: my_first_dag
My first Airflow DAG example
Schedule: @daily
Next Run: 2025-09-22, 00:00:00
Grid
Graph
Calendar
Task Duration
Task Tries
Landing Times
Gantt
Details
Code
Audit Log
23-09-2025 12:30:28
25
All Run Types
All Run States
Clear Filters
Auto-refresh
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
my_first_dag
Details
Graph
Gantt
Code
task_1
task_2
DAG Summary
Total Tasks
2
BashOperators
2
DAG Details
Dag id
my_first_dag
Description
My first Airflow DAG example
Fileloc
/opt/airflow/dags/my_first_dag.py
Has import errors
false
Has task concurrency limits
false
Show desktop





Summary of First Airflow DAG

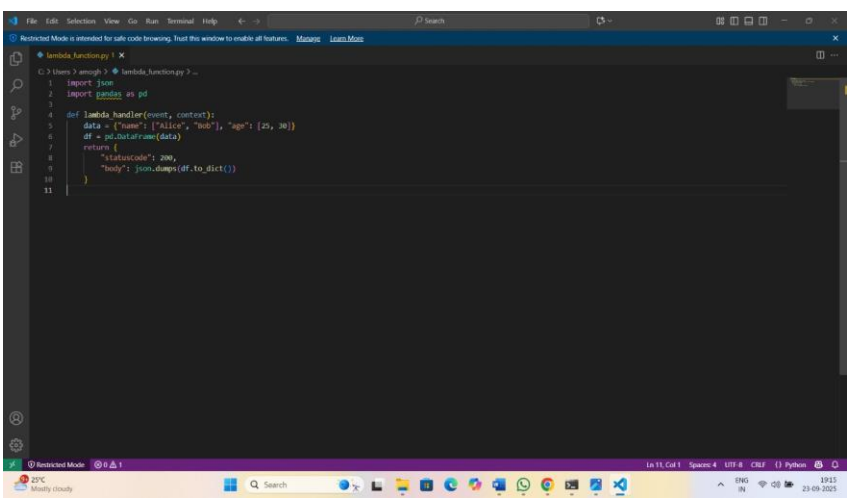
- **Objective:** Understand DAG structure.
- **Steps Done:**
  - Created my_first_dag.py inside dags/.
  - Defined DAG with @daily schedule, two BashOperator tasks.
  - Set dependency: task_1 >> task_2.
  - Verified DAG in UI, triggered run, saw successful graph.

Task 3: Troubleshooting Practice

Step 1: Prepare Your Code File

1. Create a file called lambda_function.py with this content:

```
import json

import pandas as pd


def lambda_handler(event, context):

    data = {"name": ["Alice", "Bob"], "age": [25, 30]}

    df = pd.DataFrame(data)

    return {

        "statusCode": 200,

        "body": json.dumps(df.to_dict())

    }
```



Step 2: Create First Directory (to generate error)

1. Make a folder for your first attempt:

mkdir lambda_test_error

2. Move your code file into it:

move lambda_function.py lambda_test_error/

3. Go inside the folder:

cd lambda_test_error

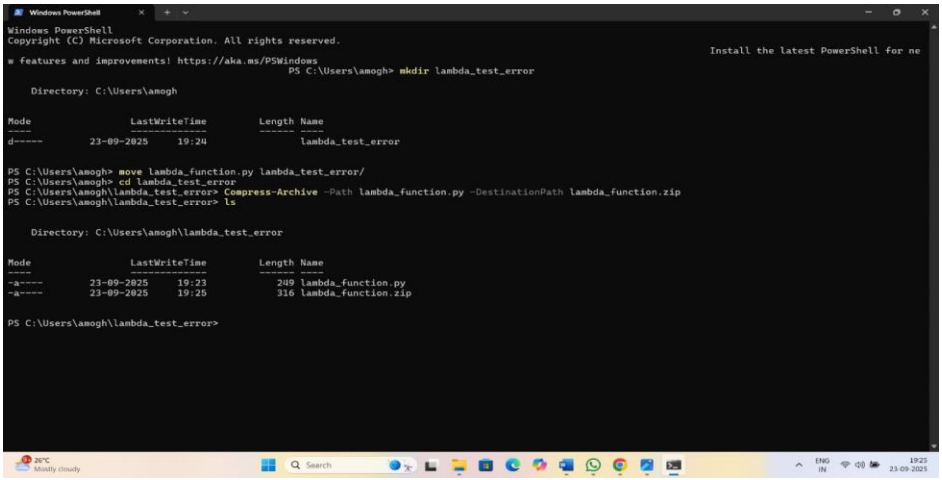
4. Zip only your code file:

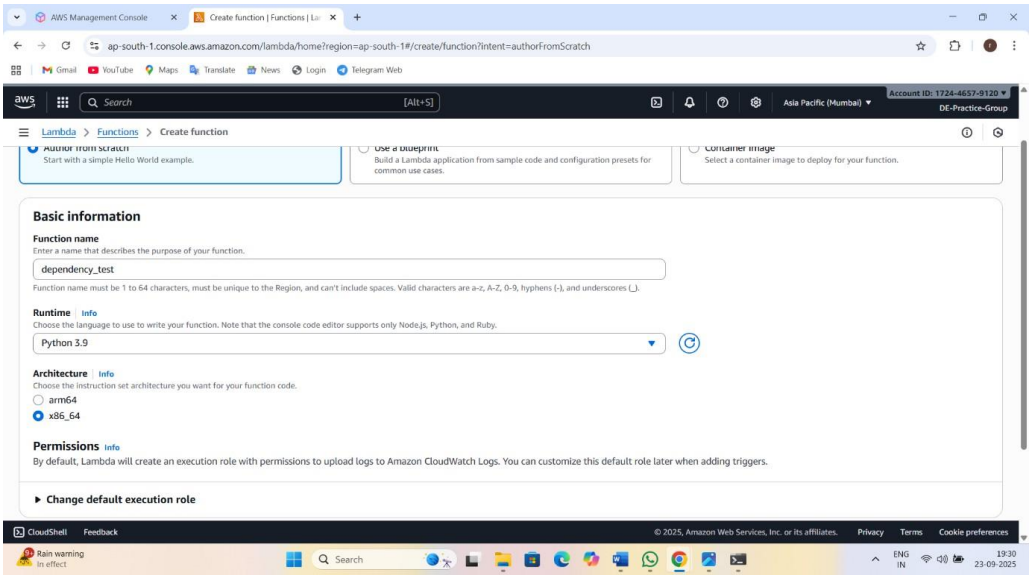
Compress-Archive -Path lambda_function.py -DestinationPath lambda_function.zip

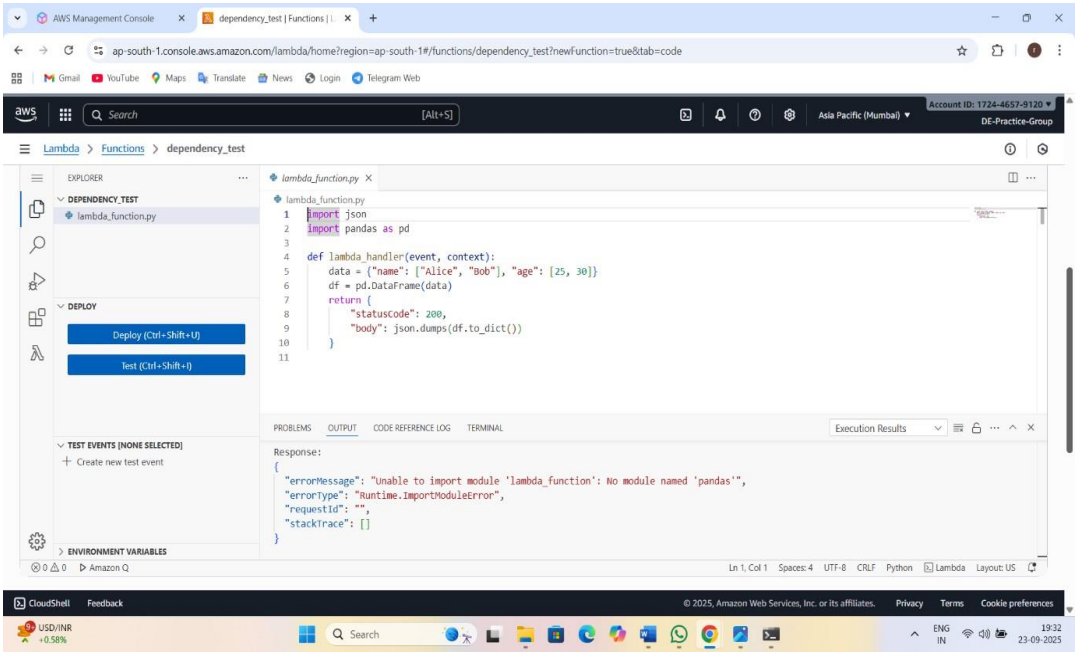
5. Upload lambda_function.zip to **AWS Lambda → Create Function → Python 3.9**.
6. Run a test event {}.

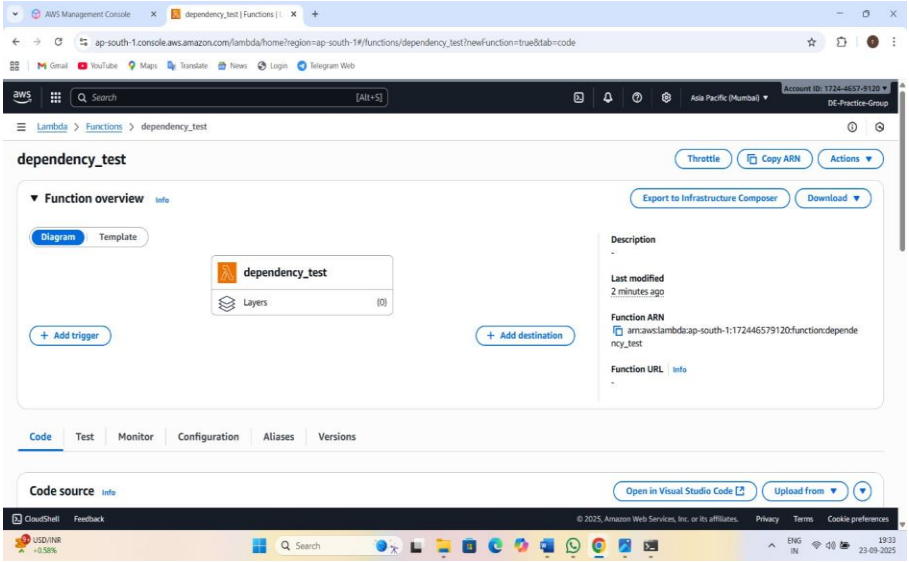
Unable to import module 'lambda_function': No module named 'pandas'

- This happens because pandas is not included in the zip.




AWS Management Console
Create function | Functions | La
ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/create/function?intent=authorFromScratch
Gmail YouTube Maps Translate News Login Telegram Web
Search [Alt+S]
Asia Pacific (Mumbai)
Account ID: 1724-4657-9120
DE-Practice-Group
Lambda > Functions > Create function
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
dependency_test
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.9
Architecture Info
Choose the instruction set architecture you want for your function code.
arm64
x86_64
Permissions Info
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.
Change default execution role
CloudShell Feedback
© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
Rain warning
In effect
Search
ENG IN
19:30
23-09-2025


AWS Management Console
dependency_test | Functions | L
ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/dependency_test?newFunction=true&tab=code
Gmail YouTube Maps Translate News Login Telegram Web
Search [Alt+S]
Asia Pacific (Mumbai)
Account ID: 1724-4657-9120
DE-Practice-Group
Lambda > Functions > dependency_test
EXPLORER
DEPENDENCY_TEST
lambda_function.py
DEPLOY
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
ENVIRONMENT VARIABLES
lambda_function.py
import json
import pandas as pd
def lambda_handler(event, context):
data = {"name": ["Alice", "Bob"], "age": [25, 30]}
df = pd.DataFrame(data)
return {
"statusCode": 200,
"body": json.dumps(df.to_dict())
}
PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL
Execution Results
Response:
{
"errorMessage": "Unable to import module 'lambda_function': No module named 'pandas'",
"errorType": "Runtime.ImportModuleError",
"requestId": "",
"stackTrace": []
}
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Lambda Layout: US
CloudShell Feedback
© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
USD/INR
+0.58%
Search
ENG IN
19:32
23-09-2025



Step 3: Create Second Directory (to fix the error)

1. Go back to your parent folder:

cd ..

2. Make a folder for the correct package:

mkdir lambda_test_success

cd lambda_test_success

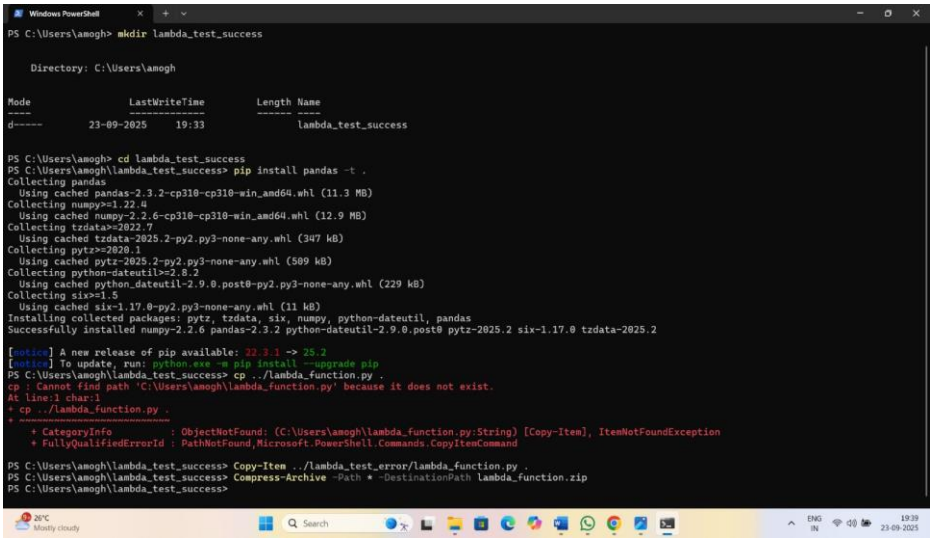
Step 4: Install Pandas in This Directory

1. Install pandas locally inside the folder:

pip install pandas -t .

2. Copy your Lambda code file into this folder:

Copy-Item ../lambda_test_error/lambda_function.py .



Step 5: Add the layer (Correct Deployment Package)

1. Go to layers tab in the function
2. there select aws managed layer
3. add pythonsdk pandas
4. then try again

{

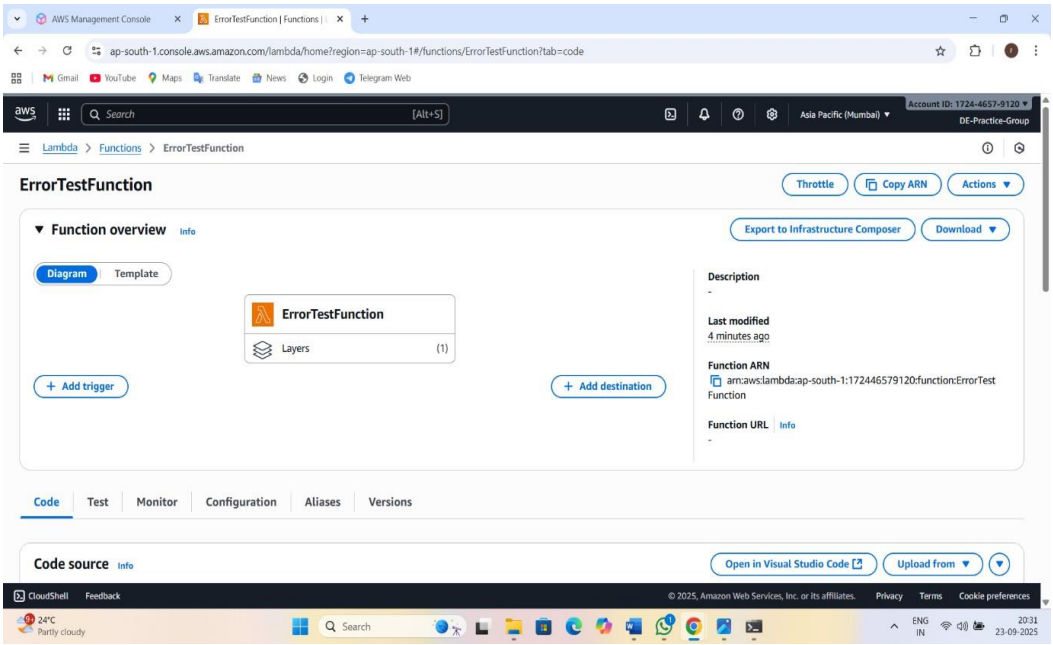
"statusCode": 200,

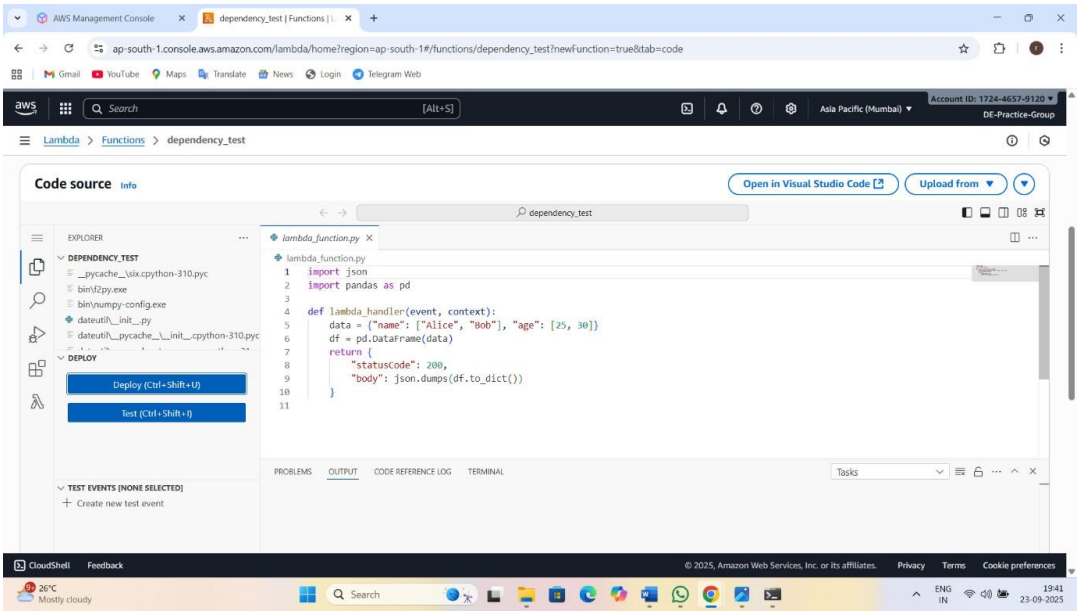
"body": "{\"name\":{\"0\":\"Alice\",\"1\":\"Bob\"},\"age\":{\"0\":25,\"1\":30}}"

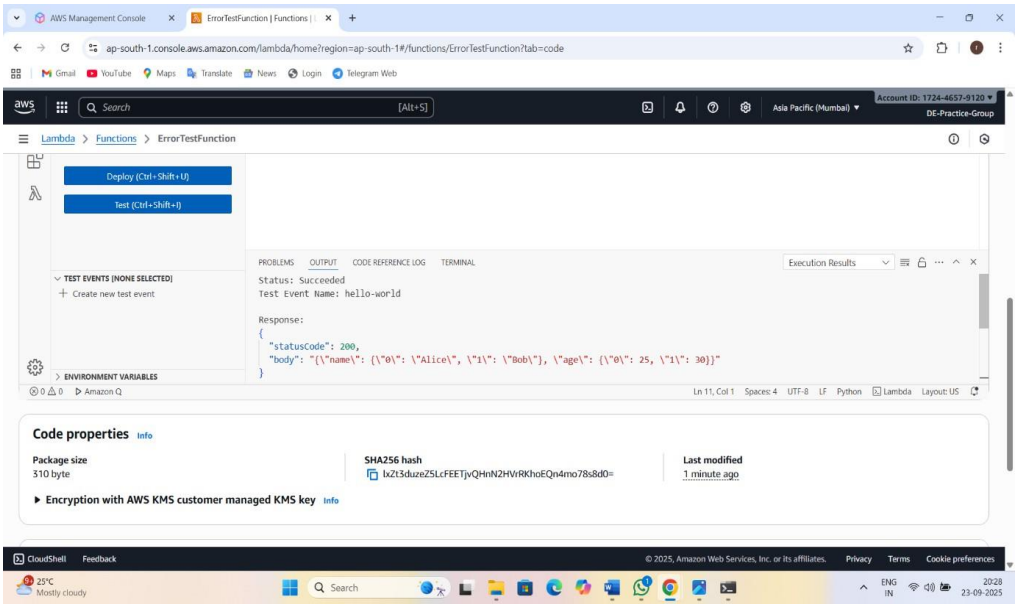
}

- Success! No import errors.

```
PS C:\Users\amogh> mkdir lambda_test_success

Compress-Archive
    The archive file 'C:\Users\amogh\lambda_test_success\lambda_function.zip' creation is in progress...
    [ooooooo
```


AWS Management Console
ErrorTestFunction | Functions |
ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/ErrorTestFunction?tab=code
Account ID: 1724-4657-9120
DE-Practice-Group
Asia Pacific (Mumbai)
Lambda > Functions > ErrorTestFunction
ErrorTestFunction
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
ErrorTestFunction
Layers (1)
Add trigger
Add destination
Description
-
Last modified
4 minutes ago
Function ARN
arn:aws:lambda:ap-south-1:172446579120:function:ErrorTest Function
Function URL
-
Code
Test
Monitor
Configuration
Aliases
Versions
Code source
Open in Visual Studio Code
Upload from
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


AWS Management Console
dependency_test | Functions |
ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/dependency_test?newFunction=true&tab=code
Account ID: 1724-4657-9120
DE-Practice-Group
Asia Pacific (Mumbai)
Lambda > Functions > dependency_test
Code source
Open in Visual Studio Code
Upload from
EXPLORER
DEPENDENCY_TEST
__pycache__\six.cpython-310.pyc
bin\f2py.exe
bin\numpy-config.exe
dateutil__init__.py
DEPLOY
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
lambda_function.py
import json
import pandas as pd
def lambda_handler(event, context):
data = {"name": ["Alice", "Bob"], "age": [25, 30]}
df = pd.DataFrame(data)
return {
"statusCode": 200,
"body": json.dumps(df.to_dict())
}
PROBLEMS
OUTPUT
CODE REFERENCE LOG
TERMINAL
Tasks
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Troubleshooting Practice (Lambda Import Error)

- **Objective:** Reproduce & fix “No module named …” error in AWS Lambda.
- **Steps Done:**
  - Created Python Lambda using requests.
  - Deployed only .py file → got **ImportModuleError: No module named 'requests'**.
  - Fixed via:
    - **Option A:** Deployment package (zip with code + library).
    - **Option B:** Created & attached a **Lambda Layer** containing requests.
  - Tested again → success (200 response with GitHub API headers).

Task 4: Research and Compare

1. AWS MWAA Pricing Analysis

Amazon MWAA offers a managed environment for running Apache Airflow, with pricing based on the environment class and additional resources like workers, schedulers, and web servers.

Estimated Monthly Cost for a Minimal MWAA Environment

- **Environment Class**: mw1.micro
- **vCPU**: 1
- **Memory**: 2 GiB
- **Pricing**: $0.055 per vCPU hour
- **Monthly Estimate**: $0.055 × 24 hours × 30 days = **$39.60**

This estimate covers the environment instance usage and does not include additional costs like worker instances, schedulers, web servers, or storage. Amazon Web Services, Inc.

---

2. Comparison: Docker-Based Local Setup vs. AWS MWAA

a. Learning/Development

- **Docker-Based Local Setup**:
  - **Pros**:
    - Quick setup using Docker Compose.
    - No cloud costs during development.
    - Ideal for testing and prototyping.
  - **Cons**:
    - Requires understanding of Docker and Airflow configurations.
    - Limited scalability and resource constraints on local machines.
    - Potential compatibility issues with certain libraries.

- **AWS MWAA**:
    - **Pros**:
        - Managed service with auto-scaling capabilities.
        - Access to AWS ecosystem and integrations.
        - No local resource constraints.
    - **Cons**:
        - Costs associated with usage.
        - Learning curve for managing AWS resources.
        - Requires internet connectivity.

b. Small-Scale Production Workload

- **Docker-Based Local Setup**:
    - **Pros**:
        - Low operational costs.
        - Full control over the environment.
    - **Cons**:
        - Limited scalability and reliability.
        - Manual setup for high availability and fault tolerance.
        - Not suitable for high-availability requirements.
- **AWS MWAA**:
    - **Pros**:
        - High availability and fault tolerance.
        - Managed scaling based on workload.
        - Integrated monitoring and logging.
    - **Cons**:
        - Ongoing operational costs.
        - Less control over the underlying infrastructure.
        - Potential vendor lock-in.

MWAA Pricing & Comparison

- **Objective:** Research AWS Managed Workflows for Apache Airflow costs and compare with Docker.
- **Steps Done:**
  - Estimated cost for a **small MWAA environment ≈ $39.60/month** (base environment + 10GB metadata DB).
  - Compared with Docker:
    - **Learning/Dev:** Docker = free, flexible, local but needs manual setup. MWAA = costly, managed, less flexible.
    - **Small Production:** Docker = cheaper, but more ops overhead. MWAA = reliable, scalable, integrated, but higher cost.